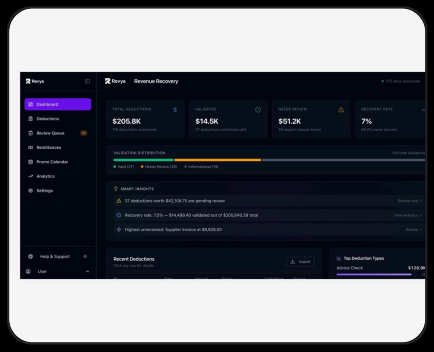
Yes. Since you want to build this **seriously for the Razorpay Buildathon**, I would build it in a very specific order:

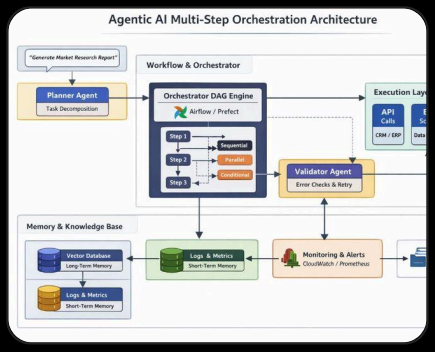
> **Backend foundation → Razorpay integration → event engine → database → AI engine → policy engine → autonomous recovery → testing/Recovery Lab → frontend → realtime dashboard → voice → deployment/live readiness.**

Do **not** start with the beautiful dashboard. The dashboard should visualize a working recovery engine, not hide a mock system.

Razorpay's current docs also support the architecture we're choosing: Test and Live webhook payload structures are intended to be the same, webhook delivery can be duplicated/out of order, signatures should be verified, and webhook handling should be fast. Razor... +1

RecoverAI — Complete Build Roadmap







PHASE 0 — Freeze the product definition

Before writing code, define exactly what we're building.

Product

RecoverAI

> Autonomous AI Revenue Recovery Agent for Razorpay merchants.

Core loop

```
Detect
  ↓
Understand
  ↓
Score risk
  ↓
Decide
  ↓
```

```
Check policy
  ↓
Act
  ↓
Verify payment
  ↓
Recover
  ↓
Learn / follow up
```

Primary recovery flow

Our first real workflow:

```
Payment fails
      ↓
RecoverAI receives event
      ↓
AI diagnoses failure
      ↓
AI calculates recovery probability
      ↓
Policy engine checks whether action is allowed
      ↓
Create Razorpay Payment Link
      ↓
Send recovery communication
      ↓
Customer pays
      ↓
Razorpay webhook
      ↓
RecoverAI verifies success
      ↓
₹X RECOVERED
```

Razorpay's Payment Link API supports creating links programmatically, and Test Mode lets you explicitly test success/failure flows. Razor... +1

PHASE 1 — Create the repository

Create:

```
recoverai/
│
├── backend/
├── ai/
├── frontend/
```

```
│
├── docs/
├── infra/
├── scripts/
│
├── docker-compose.yml
├── .gitignore
├── .env.example
└── README.md
```

Initialize Git:

```bash
git init
```

Then make your first commit:

```bash
git add .
git commit -m "chore: initialize RecoverAI monorepo"
```

PHASE 2 — Backend foundation

We build the backend first.

Stack

```
Node.js
Express.js
TypeScript
PostgreSQL
Prisma
Redis
BullMQ
Zod
Pino
JWT
```

Create:

```
backend/
├── src/
├── prisma/
├── tests/
```

```
├── package.json
```

Install:

```bash
npm init -y
npm install express cors helmet dotenv zod
npm install razorpay
npm install @prisma/client
npm install bullmq ioredis
npm install pino
npm install jsonwebtoken bcrypt
npm install -D typescript tsx nodemon prisma
npm install -D @types/node @types/express
npm install -D @types/cors @types/jsonwebtoken @types/bcrypt
```

PHASE 3 — Backend folder architecture

Build this:

```
backend/src/

├── config/
│   ├── env.ts
│   ├── database.ts
│   ├── redis.ts
│   └── razorpay.ts
│
├── middleware/
│   ├── auth.ts
│   ├── errorHandler.ts
│   ├── rateLimiter.ts
│   └── requestLogger.ts
│
├── modules/
│
│   ├── auth/
│   ├── merchants/
│   ├── razorpay/
│   ├── webhooks/
│   ├── sync/
│   ├── payments/
│   ├── recovery/
│   ├── ai/
│   ├── policies/
│   ├── actions/
│   ├── analytics/
│   ├── agent/
```

```
│   ├── audit/
│   ├── voice/
│   └── lab/
│
├── queues/
├── workers/
├── realtime/
│
├── app.ts
└── server.ts
```

PHASE 4 — Configuration

Create:

```
backend/.env
```

Initially:

env

```
NODE_ENV=development
PORT=5000

DATABASE_URL=postgresql://...

REDIS_URL=redis://localhost:6379

RAZORPAY_MODE=test

RAZORPAY_TEST_KEY_ID=
RAZORPAY_TEST_KEY_SECRET=

RAZORPAY_TEST_WEBHOOK_SECRET=

AI_SERVICE_URL=http://localhost:8000
```

Later:

env

```
RAZORPAY_LIVE_KEY_ID=
RAZORPAY_LIVE_KEY_SECRET=
RAZORPAY_LIVE_WEBHOOK_SECRET=
```

Never expose these to React.

PHASE 5 — Database

Use PostgreSQL.

Our initial Prisma schema needs these entities:

```
Merchant
MerchantConnection

Customer

Payment
Order
Subscription

WebhookEvent

RecoveryCase
RecoveryAttempt
RecoveryAction

AIDecision

Policy
PolicyDecision

AgentRun

AuditLog
```

Later:

```
CheckoutSession
PromiseToPay
VoiceInteraction

Experiment
ExperimentCase
ExperimentEvent
```

PHASE 6 — Database relationships

The important relationship is:

```
Merchant
   |
   ├── Customers
```

```
    |
    ├── Payments
    |
    ├── Orders
    |
    ├── Subscriptions
    |
    ├── RecoveryCases
    |       |
    |       ├── AIDecisions
    |       ├── RecoveryAttempts
    |       ├── RecoveryActions
    |       └── PolicyDecisions
    |
    └── AuditLogs
```

PHASE 7 — Health checks

Before Razorpay:

Backend

```http
GET /health
```

Returns:

```json
{
  "status": "ok",
  "service": "recoverai-backend"
}
```

AI

```http
GET /health
```

Returns:

```json
{
  "status": "ok",
```

```
  "service": "recoverai-ai"
}
```

Do not move forward until:

```
Frontend ❌
AI ❌
Razorpay ❌

Backend ✅
Database ✅
Redis ✅
```

PHASE 8 — Redis + BullMQ

Now create:

```
queues/

event.queue.ts
ai.queue.ts
recovery.queue.ts
notification.queue.ts
followup.queue.ts
```

Why?

Because this:

```
Razorpay
 ↓
Webhook
 ↓
AI
 ↓
Payment Link
```

should **not** happen inside the webhook request.

Instead:

```
Webhook
 ↓
Validate
 ↓
Store
 ↓
```

```
Queue
 ↓
200 OK
```

Then:

```
Worker
 ↓
AI
 ↓
Policy
 ↓
Action
```

Razorpay states webhook endpoints should respond with a 2XX response within 5 seconds, and duplicate deliveries are expected. Razor… +1

PHASE 9 — Razorpay Test integration

Now connect Razorpay.

Create:

```
modules/razorpay/

razorpay.client.ts
payments.ts
orders.ts
paymentLinks.ts
subscriptions.ts
```

Initially we only need:

```
Fetch payments
Create payment link
Fetch payment
```

Razorpay's Payments API currently supports fetching payments, while Payment Links are the mechanism we'll use to actually request recovery payment. Razor… +1

PHASE 10 — Razorpay webhook

Create:

```http
POST /api/v1/webhooks/razorpay
```

Flow:

```
Razorpay
   ↓
Webhook
   ↓
Read RAW body
   ↓
Verify signature
   ↓
Read event ID
   ↓
Check duplicate
   ↓
Store WebhookEvent
   ↓
Queue event
   ↓
200
```

Razorpay uses an HMAC-SHA256 signature and the `x-razorpay-event-id` header can be used for duplicate detection. 1 Razor...

PHASE 11 — Webhook events

Start with the smallest useful set.

Payments

```
payment.authorized
payment.captured
payment.failed
```

Later add:

```
payment.dispute.created
```

and relevant subscription/payment-link events as required.

PHASE 12 — Event normalizer

Do not let the rest of your system depend directly on Razorpay's raw payload.

Create:

```
event.normalizer.ts
```

Convert:

```
Razorpay event
```

into:

```
JSON
```

```json
{
  "source": "razorpay",
  "type": "PAYMENT_FAILED",
  "merchantId": "...",
  "paymentId": "...",
  "amount": 499900,
  "currency": "INR",
  "customerId": "...",
  "occurredAt": "..."
}
```

Now the recovery engine doesn't care whether the event came from:

```
Razorpay
Recovery Lab
future payment provider
```

PHASE 13 — Historical synchronization

This is **very important for your idea**.

When merchant turns RecoverAI on:

```
Merchant connects Razorpay
       ↓
RecoverAI syncs historical data
       ↓
Payments
Orders
Subscriptions
       ↓
Database
```

```
        ↓
AI analyzes
        ↓
Cases created
        ↓
Dashboard populated
```

Razorpay's Payments API supports fetching payments with date ranges and pagination using `from`, `to`, `count`, and `skip`; the current documented maximum `count` is 100. Razor...

So if the merchant already has:

```
10,000 payments
```

we don't hardcode:

```
₹2,41,000 at risk
```

We calculate it from actual synchronized records.

PHASE 14 — Historical classification

For each historical payment:

```
SUCCESS
FAILED
AUTHORIZED
REFUNDED
```

AI/backend determines:

```
Should this become a recovery case?
```

Example:

```
Payment:
₹5,000

Status:
failed

Customer:
7 previous successful payments
```

```
Previous recovery:
0

Time:
2 hours ago

Result:

AT_RISK
```

But:

```
Payment:
₹5,000

Status:
failed

Customer:
0 history

Age:
180 days

Result:

DO_NOT_RECOVER
```

This is where your system starts becoming intelligent rather than simply reacting to every failed payment.

PHASE 15 — Recovery Case Engine

Create:

```
recovery/
├── recovery.service.ts
├── case.engine.ts
├── state.machine.ts
└── recovery.controller.ts
```

States:

```
NEW
 ↓
AT_RISK
 ↓
ANALYZING
```

```
 ↓
PLANNED
 ↓
POLICY_CHECK
 ↓
RECOVERING
 ↓
WAITING
 ↓
RECOVERED
```

Alternative:

```
BLOCKED
ESCALATED
FAILED
STOPPED
```

PHASE 16 — Build the stopping rules BEFORE autonomous AI

This is crucial.

Your AI must not be allowed to spam customers.

Example policy:

```
Maximum automatic recovery attempts = 3

Minimum cooldown = 6 hours

Maximum automatic amount = ₹25,000

Maximum customer contacts = 3

After 3 failures → STOP

High-value payment → HUMAN REVIEW

Customer opted out → STOP

Already paid → STOP

Refunded → STOP
```

This directly addresses:

> compliant escalation + stopping rules.

PHASE 17 — Build the AI service

Now we build AI.

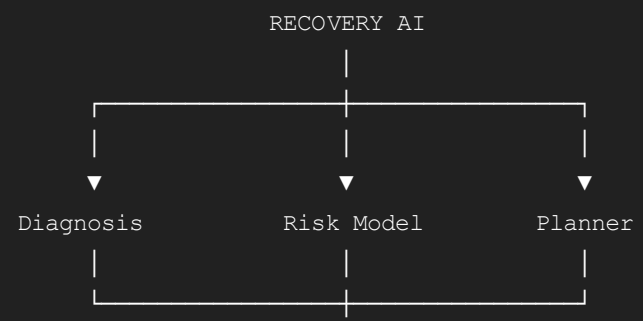
Create:

```
ai/
├── app/
│   ├── main.py
│   │
│   ├── api/
│   │   └── routes/
│   │
│   ├── agents/
│   │
│   ├── reasoning/
│   │
│   ├── prompts/
│   │
│   ├── models/
│   │
│   ├── safety/
│   │
│   └── providers/
│
└── tests/
```

PHASE 18 — AI architecture

Don't build one giant "AI agent."

Build specialized components.



PHASE 19 — AI Diagnosis

Input:

```json
{
  "payment": {},
  "customer": {},
  "history": {},
  "previousAttempts": []
}
```

Output:

```json
{
  "diagnosis": "temporary_payment_failure",
  "confidence": 0.91
}
```

Possible diagnoses:

```
TEMPORARY_PAYMENT_FAILURE
CUSTOMER_INACTIVE
REPEATED_FAILURE
HIGH_VALUE_FAILURE
UNKNOWN
```

PHASE 20 — Risk scoring

Build a scoring engine.

Example:

```
Recovery Score
=
customer history
+
payment recency
+
amount
```

```
+
failure frequency
+
payment method
+
previous recovery success
```

Output:

```
JSON

{
  "riskScore": 82,
  "recoveryProbability": 0.87
}
```

Don't claim that this is a scientifically calibrated probability initially.

Call it:

```
recovery probability estimate
```

until we have evaluation data.

PHASE 21 — AI Planner

Planner outputs only allowed actions:

```
NO_ACTION

PAYMENT_LINK

FOLLOW_UP

VOICE_CALL

ESCALATE

STOP
```

Example:

```
JSON

{
  "recommendedAction": "PAYMENT_LINK",
  "confidence": 0.91,
```

```
  "reason": "Customer has strong successful payment history."
```

PHASE 22 — Structured AI output

Never allow:

```
AI:
"Hey maybe send a payment link..."
```

Instead require schema:

JSON

```
{
  "diagnosis": "...",
  "recoveryProbability": 0.87,
  "recommendedAction": "PAYMENT_LINK",
  "confidence": 0.91,
  "riskFlags": [],
  "reason": "..."
}
```

Then validate it with Pydantic/Zod.

PHASE 23 — AI safety validator

Create:

```
ai/app/safety/output_validator.py
```

It checks:

```
Is action allowed?
Is confidence sufficient?
Is amount within range?
Are required fields present?
Does action exist?
Are risk flags present?
```

If invalid:

```
AI → REJECTED
```

not:

```
AI → execute anyway
```

PHASE 24 — Policy engine

Now combine:

```
AI
+
Merchant policy
+
Payment state
+
Recovery state
```

Example:

```
AI recommends:
PAYMENT_LINK

Amount:
₹4,999

Merchant maximum:
₹25,000

Attempts:
0

Customer opted out:
No

Payment already succeeded:
No

Result:
APPROVED
```

PHASE 25 — Action engine

Create:

```
actions/
├── action.engine.ts
├── paymentLink.action.ts
├── notification.action.ts
├── followup.action.ts
└── escalation.action.ts
```

The engine executes only approved actions.

PHASE 26 — First actual recovery

This is our **MVP milestone**.

```
payment.failed
      ↓
Recovery Case
      ↓
AI
      ↓
Policy
      ↓
Payment Link
```

Razorpay currently provides `POST /v1/payment_links` for creating Standard Payment Links. Razor...

PHASE 27 — Recovery verification

Creating a Payment Link does **not** mean recovered.

This distinction is extremely important.

Wrong:

```
Payment Link created
=
₹5,000 recovered
```

Correct:

```
Payment Link created
=
₹5,000 recovery attempt
```

Only:

```
payment.captured
```

means:

```
₹5,000 RECOVERED
```

PHASE 28 — Money metrics

Create calculations:

```
Gross at risk
Recovered revenue
Recovery attempts
Recovery rate
Average recovery time
Recovery value
Escalated amount
Blocked amount
```

For example:

```
AT RISK
₹75,000

RECOVERED
₹24,000

RECOVERY RATE
32%

ACTIVE CASES
17
```

All calculated from database events.

PHASE 29 — Audit system

Every important event becomes:

```
AuditLog
```

Example:

```
14:21:03
PAYMENT_FAILED

14:21:04
CASE_CREATED

14:21:04
AI_ANALYSIS

14:21:04
POLICY_APPROVED

14:21:05
PAYMENT_LINK_CREATED

14:27:31
PAYMENT_CAPTURED

14:27:31
RECOVERY_CONFIRMED
```

This directly supports the buildathon's auditability requirement.

PHASE 30 — Autonomous Agent

Now we make it automatic.

Agent states:

```
DISABLED
PAUSED
SYNCING
ACTIVE
ERROR
```

When:

```
ACTIVE
```

the agent automatically:

```
receive event
 ↓
analyze
 ↓
```

```
decide
 ↓
policy
 ↓
action
 ↓
verify
 ↓
follow-up
```

No dashboard button required.

PHASE 31 — Merchant enable flow

When merchant first connects:

```
CONNECT RAZORPAY
       ↓
VERIFY CREDENTIALS
       ↓
CHOOSE MODE
       ↓
HISTORICAL SYNC
       ↓
AI ANALYSIS
       ↓
CREATE EXISTING CASES
       ↓
AGENT READY
```

Then:

```
[ ENABLE AUTONOMOUS RECOVERY ]
```

When enabled:

```
ACTIVE
```

From then onward, new events are processed automatically.

PHASE 32 — TEST MODE

Now build actual Razorpay Test Mode.

You will have:

```
RAZORPAY_MODE=test
```

Test credentials.

Test webhook.

Test payment.

Razorpay documents that Test Mode webhook payload structure matches Live Mode, which is exactly why we'll use Test Mode to validate the same architecture. 1 Razor...

PHASE 33 — Local webhook

Razorpay needs a publicly reachable webhook endpoint; localhost directly cannot receive webhook delivery. Their current documentation recommends using a public staging environment or an appropriate tunnel for local testing. 1 Razor...

For development:

```
Your computer
      ↓
public HTTPS tunnel
      ↓
localhost:5000
```

Later:

```
Internet
 ↓
Production backend
```

PHASE 34 — Test the complete real flow

Do not use fake success.

Run:

```
Razorpay Test
      ↓
Payment fails
      ↓
Webhook
      ↓
```

```
Database
      ↓
AI
      ↓
Policy
      ↓
Payment Link
      ↓
Open Test Payment Link
      ↓
Select SUCCESS
      ↓
Webhook
      ↓
RECOVERED
```

Razorpay's current Payment Link testing flow lets you choose a success or failure outcome on the Test Payment Link. 1 Razor...

PHASE 35 — Failure testing

Now intentionally break things.

Test:

```
Duplicate webhook
```

Expected:

```
Ignored
```

Test:

```
Webhook out of order
```

Expected:

```
Correct final state
```

Razorpay explicitly warns that webhook events may not arrive in order. 1 Razor...

Test:

```
AI unavailable
```

Expected:

```
Case remains safe
Retry later
No uncontrolled action
```

Test:

```
Payment already successful
```

Expected:

```
STOP
```

Test:

```
₹100,000 recovery
```

Expected:

```
HUMAN REVIEW
```

PHASE 36 — Recovery Lab

Only after the real Razorpay flow works.

Create:

```
Recovery Lab
```

The lab generates cases:

```
Payment failures
Subscription failures
Checkout abandonment
Overdue cases
```

But they go through:

```
SAME CASE ENGINE
SAME AI
SAME POLICY ENGINE
```

```
SAME ACTION ENGINE
```

This is important.

Don't make:

```
Lab engine ≠ Real engine
```

Make:

```
Lab
  ↓
Event Simulator
  ↓
SAME EVENT PIPELINE
```

PHASE 37 — Dynamic batches

The judge selects:

```
Cases: 100

Payment failures: 40
Subscriptions: 20
Checkout: 20
Receivables: 20

Seed: 482912
```

Then:

```
RUN
```

The system generates cases dynamically.

PHASE 38 — Never fake the batch

Store:

```
Experiment
```

with:

```
experimentId
seed
agentVersion
policyVersion
createdAt
caseCount
```

Each case:

```
ExperimentCase
```

Each event:

```
ExperimentEvent
```

Then the judge can inspect it.

PHASE 39 — Recovery Lab replay

Add:

```
REPLAY
```

Judge can click:

```
Experiment #001

[ VIEW CASES ]

[ REPLAY ]

[ COMPARE ]

[ EXPORT AUDIT ]
```

PHASE 40 — Failure injection Lab

Add:

```
☐ Duplicate webhook

☐ Out-of-order events
```

```
☐ AI timeout

☐ Razorpay timeout

☐ Notification failure

☐ Customer pays late

☐ Customer already paid

☐ Maximum attempts exceeded
```

Now judges can actively break your system.

That's much more impressive than a static demo.

PHASE 41 — Now build frontend

Only now.

Frontend:

```
React
TypeScript
Vite
Tailwind
shadcn/ui
Framer Motion
Recharts
Lucide
```

Theme:

```
Background:
#0B0907

Primary gold:
#D99A35

Bright gold:
#E9B45B

Primary text:
#F5EFE3

Secondary:
#A8A19A
```

```
Cards:
#15120F

Borders:
#2A251F
```

PHASE 42 — Frontend structure

```
frontend/src/

├── components/
│   ├── ui/
│   ├── charts/
│   ├── dashboard/
│   ├── recovery/
│   ├── agent/
│   ├── lab/
│   └── voice/
│
├── pages/
│   ├── Login
│   ├── Onboarding
│   ├── Dashboard
│   ├── RecoveryCases
│   ├── RecoveryDetails
│   ├── Agent
│   ├── Customers
│   ├── Subscriptions
│   ├── CheckoutRecovery
│   ├── VoiceRecovery
│   ├── Policies
│   ├── RecoveryLab
│   ├── AuditTrail
│   ├── Integrations
│   └── Settings
│
├── services/
├── hooks/
├── store/
├── types/
├── utils/
│
├── App.tsx
└── main.tsx
```

PHASE 43 — Login

Keep it simple.

```
RECOVERAI

Autonomous Revenue Recovery

Email
Password

[ SIGN IN ]
```

Then onboarding.

PHASE 44 — Merchant onboarding

```
CONNECT RAZORPAY

Choose environment

○ Test Mode
○ Live Mode

Razorpay Key ID
••••••••••

Razorpay Secret
••••••••••

Webhook Secret
••••••••••

[ TEST CONNECTION ]
```

Do not display secrets after saving.

PHASE 45 — Agent activation

Show:

```
Razorpay Connected ✓

Historical data
12,481 payments
```

Then:

```
INITIAL ANALYSIS COMPLETE

₹1,84,200
Potential revenue at risk

243
Recovery candidates

37
High priority

[ ENABLE AUTONOMOUS RECOVERY ]
```

Again, these are calculated.

PHASE 46 — Main dashboard

The dashboard should immediately communicate:

```
How much money was at risk?
How much did AI recover?
How many cases are active?
What is AI doing right now?
```

Top:

```
┌──────────────────────────────────────────┐
│ RECOVERED                                │
│ ₹24,820                                  │
│ +18.4%                                   │
└──────────────────────────────────────────┘

┌──────────────────────────────────────────┐
│ AT RISK                                  │
│ ₹74,200                                  │
└──────────────────────────────────────────┘

┌──────────────────────────────────────────┐
│ ACTIVE CASES                             │
│ 37                                       │
└──────────────────────────────────────────┘
```

PHASE 47 — Live agent activity

Right side:

```
AGENT ACTIVITY

• 20:32 Payment failed
  ₹4,999

  Diagnosing...

• 20:32 Recovery approved

  Payment Link created

• 20:34 Customer opened link

• 20:35 Payment captured

  ✓ ₹4,999 recovered
```

This should update live.

PHASE 48 — Realtime

Implement SSE:

```
Backend
   ↓
SSE
   ↓
React
```

Events:

```
PAYMENT_FAILED
CASE_CREATED
AI_STARTED
AI_COMPLETED
POLICY_APPROVED
ACTION_EXECUTED
PAYMENT_RECOVERED
AGENT_STATUS
```

PHASE 49 — Recovery Cases page

Filters:

```
All
At Risk
Recovering
Recovered
Escalated
Stopped
```

Columns:

```
Customer
Amount
Problem
Risk
AI action
Status
Last action
```

PHASE 50 — Case details

This page should be one of your strongest UI pages.

Show:

```
₹4,999
PAYMENT FAILED

Recovery probability
87%

AI diagnosis
Temporary payment failure

Recommended action
Payment Link

Policy
APPROVED

Attempts
1 / 3
```

Then timeline.

PHASE 51 — Agent page

Show:

```
AUTONOMOUS AGENT

● ACTIVE

Processed today
218

Recovered
₹24,820

Actions
173

Escalations
12

Stopped
33

[ PAUSE AGENT ]

[ DISABLE AGENT ]
```

PHASE 52 — Policy page

Merchant controls:

```
AUTONOMOUS RECOVERY

Maximum automatic recovery
₹25,000

Maximum attempts
3

Cooldown
6 hours

High-value threshold
₹25,000

Customer contact limit
3
```

```
Require human review
Above threshold

[ SAVE POLICY ]
```

This makes the system feel like a real merchant product.

PHASE 53 — Audit Trail

Display:

```
TIMESTAMP
EVENT
CASE
ACTOR
DECISION
RESULT
```

Example:

```
20:32:01
AI_DECISION
RC-182
RecoverAI
PAYMENT_LINK
Approved

20:32:02
ACTION_EXECUTED
RC-182
Agent
PAYMENT_LINK
Success

20:35:18
PAYMENT_CAPTURED
RC-182
Razorpay
₹4,999
Success
```

PHASE 54 — Recovery Lab UI

This becomes the judge's playground.

```
RECOVERY LAB

Cases
[100]

Seed
[482912]

Failure injection

☑ Duplicate webhook
☐ AI timeout
☐ Late payment
☐ API timeout

Policy

Max attempts [3]
Max amount [25000]

[ RUN EXPERIMENT ]
```

Then results.

PHASE 55 — Add voice recovery

Only after everything above works.

Architecture:

```
Voice
 ↓
Speech-to-text
 ↓
AI
 ↓
Intent
 ↓
Promise-to-Pay
 ↓
Policy
 ↓
Follow-up
```

Example:

Customer:

> "Bhai kal payment kar dunga."

System:

```
Intent:
PROMISE_TO_PAY

Date:
Tomorrow

Amount:
₹8,000

Confidence:
94%
```

Then:

```
Promise tracker
 ↓
Tomorrow
 ↓
Payment check
 ↓
If unpaid
 ↓
Follow-up
```

This becomes your memorable **Hinglish Voice Recovery** feature.

PHASE 56 — 3D decision

You asked whether we should make it a 3D website.

My recommendation:

Do NOT make the entire dashboard 3D.

Financial dashboards need:

```
clarity
speed
trust
data density
```

Instead use **selective 3D**:

Hero

A subtle animated:

```
Revenue recovery orbit
```

or:

```
glowing gold financial network
```

Agent page

A small:

```
3D AI agent core
```

with particles representing active recovery cases.

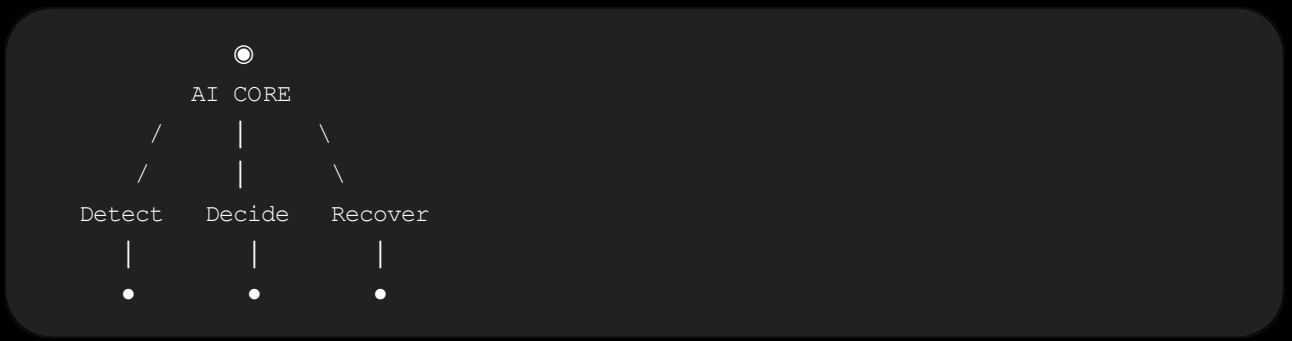
Everything else:

```
2D premium dashboard
```

That will look much more professional.

PHASE 57 — AI visual

The Agent page can show:



Cases flow toward the center.

This gives you the "wow" factor without making the product gimmicky.

PHASE 58 — Test everything

Create automated tests.

Backend:

```
webhook.test.ts
idempotency.test.ts
recovery.test.ts
policy.test.ts
action.test.ts
state-machine.test.ts
```

AI:

```
diagnosis.test.py
planner.test.py
safety.test.py
```

Frontend:

```
dashboard.test.tsx
recovery.test.tsx
agent.test.tsx
```

PHASE 59 — Critical test scenarios

You should be able to demonstrate:

Test 1

```
Failed payment
→ recovered
```

Test 2

```
Failed payment
→ payment link
→ customer doesn't pay
→ follow-up
```

Test 3

```
Failed payment
→ three attempts
→ STOP
```

Test 4

```
High-value payment
→ HUMAN REVIEW
```

Test 5

```
Duplicate webhook
→ one case
```

Test 6

```
Out-of-order webhook
→ correct final state
```

Test 7

```
AI unavailable
→ safe retry
```

Test 8

```
Customer already paid
→ STOP recovery
```

Test 9

```
Agent disabled
→ no new actions
```

Test 10

```
Historical failed payments
→ analyzed after enable
```

PHASE 60 — Live mode

Only after Test Mode is stable.

Architecture doesn't change:

```
TEST

Razorpay Test
     ↓
RecoverAI
     ↓
Test Database


LIVE

Razorpay Live
     ↓
RecoverAI
     ↓
Live Database
```

Use separate credentials and webhook secrets.

Do not mix them.

PHASE 61 — Live safety modes

Add three modes:

Monitor

```
AI analyzes
AI recommends
NO action
```

Assisted

```
AI analyzes
Policy checks
Merchant approves
Action executes
```

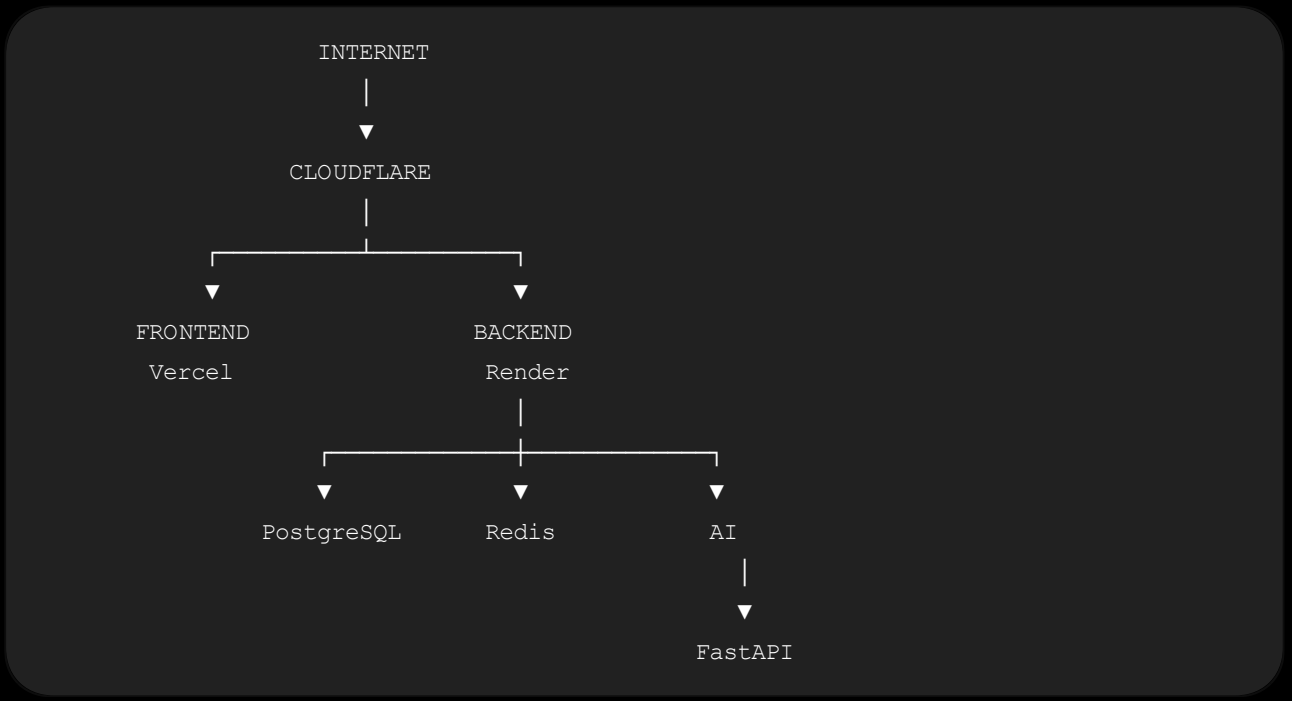
Autonomous

```
AI analyzes
Policy checks
Action automatically executes
```

This is an excellent product feature because a real merchant should be able to build trust progressively.

PHASE 62 — Live deployment

Eventually:



You can use other hosting providers too; the architecture matters more than the specific provider.

PHASE 63 — Live Razorpay

Once everything is tested:

```
RAZORPAY_MODE=live
```

Then configure:

```
Live API credentials
Live webhook
Live webhook secret
HTTPS endpoint
```

Razorpay recommends validating and testing webhooks before going live. Razor... +1

For the buildathon, **do not depend on Live Mode to prove your project**. Your Test Mode integration should be real and demonstrable; Live Mode should be a supported

deployment path.

PHASE 64 — Documentation

Create:

```
docs/

01-product.md
02-architecture.md
03-database.md
04-backend.md
05-razorpay.md
06-webhooks.md
07-ai.md
08-policy-engine.md
09-recovery-engine.md
10-autonomous-agent.md
11-realtime.md
12-recovery-lab.md
13-voice.md
14-security.md
15-testing.md
16-deployment.md
17-live-mode.md
18-demo.md
```

PHASE 65 — Buildathon documentation

Your README should answer:

What problem?

Revenue disappears after failed payments, abandonment and overdue transactions.

What did we build?

Autonomous AI revenue recovery.

How does it work?

```
Detect → Diagnose → Decide → Act → Verify → Recover
```

Why AI?

AI understands customer/payment context and chooses the most appropriate bounded intervention.

Why not let AI directly move money?

Policy engine and action allowlist constrain the AI.

How do you measure success?

```
Recovered revenue
Recovery rate
Recovery time
Attempt success
Escalation rate
```

How is it safe?

```
Policy limits
Stopping rules
Idempotency
Audit trail
Human escalation
```

PHASE 66 — What judges should see

Your five-minute demo should be:

0:00–0:30

Problem.

> "Revenue doesn't disappear only when a payment fails. It disappears when nobody closes the loop."

0:30–1:15

Dashboard.

Show:

```
₹X at risk
₹Y recovered
```

1:15–2:00

Trigger actual Test Mode failure.

```
payment.failed
```

Dashboard immediately shows:

```
NEW CASE
```

2:00–2:45

AI:

```
Diagnosis
Risk
Recommendation
Policy
```

Then:

```
Payment Link created
```

2:45–3:15

Make Test Payment.

Then:

```
✓ PAYMENT CAPTURED
₹4,999 RECOVERED
```

3:15–4:00

Show:

```
Audit Trail
```

and:

```
Policy
```

4:00–4:40

Recovery Lab.

Run:

```
100 dynamic cases
```

Show:

```
₹X at risk
₹Y recovered
```

Then change:

```
Max attempts
```

and run again.

Show different result.

4:40–5:00

Hinglish Voice Recovery.

> "Bhai kal payment kar dunga."

System:

```
Promise-to-Pay detected
Tomorrow
₹8,000
```

Finish with:

> **"RecoverAI doesn't just identify lost revenue. It closes the recovery loop."**

FINAL BUILD ORDER

This is the order I want you to actually follow:

```
01  Repository
 ↓
02  Backend foundation
 ↓
03  PostgreSQL
 ↓
04  Prisma schema
 ↓
05  Redis
 ↓
06  BullMQ
 ↓
07  Backend health
 ↓
08  Razorpay client
 ↓
09  Razorpay Test credentials
 ↓
10  Webhook endpoint
 ↓
11  Signature verification
 ↓
12  Idempotency
 ↓
13  Event normalization
 ↓
14  Historical sync
 ↓
15  Payment storage
 ↓
16  Recovery Case engine
 ↓
17  State machine
 ↓
18  Stopping rules
 ↓
19  AI service
 ↓
20  AI diagnosis
 ↓
21  Risk scoring
 ↓
22  AI planner
 ↓
23  Structured output
 ↓
24  AI safety validation
```

```
 ↓
25  Policy engine
 ↓
26  Action engine
 ↓
27  Payment Link recovery
 ↓
28  Payment verification
 ↓
29  Recovered revenue calculation
 ↓
30  Audit trail
 ↓
31  Autonomous agent
 ↓
32  Test Mode end-to-end
 ↓
33  Failure testing
 ↓
34  Recovery Lab
 ↓
35  Dynamic batches
 ↓
36  Failure injection
 ↓
37  Frontend foundation
 ↓
38  Login/onboarding
 ↓
39  Dashboard
 ↓
40  Recovery cases
 ↓
41  Agent page
 ↓
42  Policy page
 ↓
43  Audit page
 ↓
44  Recovery Lab UI
 ↓
45  Realtime SSE
 ↓
46  Premium black/gold design
 ↓
47  Selective 3D
 ↓
48  Hinglish Voice Recovery
 ↓
49  Automated tests
 ↓
50  Deployment
```

```
 ↓
51  Live readiness
 ↓
```

The most important rule

Don't build 52 things simultaneously.

Build in these **7 vertical milestones**:

Milestone	Must work
M1	Backend + DB + Redis + AI health
M2	Razorpay Test → webhook → DB
M3	Failed payment → AI → policy → Payment Link
M4	Payment Link → payment captured → recovered money
M5	Autonomous agent + historical sync + audit
M6	Recovery Lab + dynamic batches + failure injection
M7	Premium frontend + realtime + voice + final demo

M4 is the critical checkpoint. Once you can cause a genuine Razorpay Test payment failure, have RecoverAI autonomously decide what to do, execute a bounded recovery action, and then see the successful payment come back through Razorpay and change the case to **RECOVERED**, you have the core of the actual product.

The current Razorpay documentation supports using Test Mode for this kind of validation: Test webhook payload structures are designed to match Live, Payment Links can be tested with success/failure outcomes, and the Payments API can be used to retrieve historical payments for synchronization. Razor… +2

So our first coding session should be M1 only: Backend + PostgreSQL + Redis + separate FastAPI AI service. Once that foundation is running, we move to M2 and connect the real Razorpay Test environment.